Práctica 2: WebRTC

Grupo: 19
Mónica Ortega Menéndez
Jorge Velasco de María
Andrés Roques Sánchez
Adrián Modino Merino

Índice del contenido

1. Resumen de las tareas realizadas 5
2. Prueba aplicaciones WebRTC 6
3. Análisis Discord 14
4. Despliegue de una aplicación WebRTC 22

Índice de figuras

Ilustración 1. Videoconferencia app 1 meet.jit.si 8
Ilustración 2. Multiconferencia app 1 meet.jit.si y encuesta 8
Ilustración 3.Guardar pizarra app 1 meet.jit.si 9
Ilustración 4. Invitar a más personas app 1 meet.jit.si 9
Ilustración 5. Videoconferencia app 2 whereby.com 10
Ilustración 6. Multiconferencia app 2 whereby.com 10
Ilustración 7. Pantalla compartida app 2 whereby.com 11
Ilustración 8. Pantalla compartida app 3 meet.google.com 11
Ilustración 9. Subtítulos app 3 meet.google.com 12
Ilustración 10. Multiconferencia app 3 meet.google.com 12
Ilustración 11. Interfaz de usuario de llamada de discord y servicios que ofrece 14
Ilustración 12. Presentación de about:webrtc 15
Ilustración 13. Establecimiento de la llamada via discord 15
Ilustración 14. Interfaz de usuario en discord durante una llamada 16
Ilustración 15. Dirección de los servidores STUN empleados 16
Ilustración 16. Tiempo de establecimiento de llamada 19
Ilustración 17. Estadisticas ICE 19
Ilustración 18. Gráficas de videollamada 21
Ilustración 19. Index.html 22
Ilustración 20. getUserMedia.html 23
Ilustración 21. LocalPeerConnection en la consola de comandos 24
Ilustración 22. LocalPeerConnection.html, Página de inicio 24
Ilustración 23.LocalPeerConnection.html, Start 25
Ilustración 24. LocalPeerConnection.html, Call 25
Ilustración 25. LocalPeerConnection.html. Hang Up 25
Ilustración 26. dataChannel en la consola de comandos 26
Ilustración 27. dataChannel.html 26
Ilustración 28. Página /cap5/ en un equipo 27
Ilustración 29. SDP en el terminal 28
Ilustración 30. Página /cap5/ en remoto 28
Ilustración 31. Cap5. Candidatos ICE seleccionados 29
Ilustración 32. Cap5. Candidatos ICE no procesados 29
Ilustración 33. Flujo de audio entrante 31
Ilustración 34. Flujo de audio saliente 32
Ilustración 35. Flujo de video saliente 32
Ilustración 36. Flujo de video saliente 33

Índice de tablas

Tabla 1. Comparación de aplicaciones 6
Tabla 2. Terminales utilizados en la práctica 7

1. Resumen de las tareas realizadas

Durante esta práctica, nos hemos adentrado en el funcionamiento y análisis de aplicaciones basadas en tecnología WebRTC (Web Real-Time Communication), con el objetivo de comprender tanto su comportamiento como su arquitectura. Iniciamos el trabajo explorando Talkroom.io, una aplicación WebRTC que permite realizar videollamadas y multiconferencias en tiempo real. Probamos todas sus funcionalidades, incluyendo el establecimiento de llamadas, compartir pantalla, uso del chat y encuestas, además de capturar y analizar los datos de tráfico generados durante su uso.

Posteriormente, realizamos una comparativa entre diferentes plataformas similares: meet.jit.si, talk.io y whereby.com, valorando aspectos como la calidad de audio y vídeo, facilidad de uso y funcionalidades adicionales. A partir de esta comparativa, seleccionamos las dos plataformas que más nos han gustado en cuanto a funcionalidades, meet.jit.si y talk.io, sobre las cuales hicimos una evaluación más exhaustiva.

En estas dos aplicaciones, examinamos en detalle los distintos servicios ofrecidos, desde llamadas uno a uno hasta multiconferencias, pasando por funciones colaborativas como compartir pantalla, encuestas e interacciones en tiempo real.

También incluimos en el análisis a Discord, una aplicación de comunicación popular que, aunque no está basada estrictamente en WebRTC en su totalidad, incorpora muchas de sus funcionalidades. Exploramos los distintos servicios que ofrece, como canales de voz, videollamadas, compartición de pantalla y su sistema de gestión de servidores.

A nivel técnico, uno de los apartados más relevantes fue la captura y análisis del tráfico entre dos dispositivos durante el establecimiento de una conexión WebRTC. Esta observación nos ayudó a entender el rol de los protocolos implicados, el intercambio de información SDP y el uso de STUN/TURN.

Como finalización de la práctica, procedimos a desplegar nuestra propia aplicación WebRTC, utilizando tecnologías como *Node.js*, *socket.io* y *node-static*. Este despliegue fue realizado dentro de un contenedor Docker. El proceso incluyó la instalación del entorno desde terminal, la prueba de captura y visualización de vídeo y, finalmente, el establecimiento de llamadas entre dos navegadores.

2. Prueba aplicaciones WebRTC

Completar con una pequeña introducción de lo que se va a presentar en esta sección.

Se realizará un análisis detallado del establecimiento y desarrollo de una videollamada WebRTC utilizando la plataforma Talkroom.io. Para ello, se emplean tres herramientas principales:

- La consola JavaScript del navegador, que permite visualizar trazas del código en ejecución y las funciones WebRTC utilizadas.
- La página interna de WebRTC del navegador (chrome://webrtc-internals/ en Chrome y about:webrtc en Firefox), que proporciona estadísticas y detalles del tráfico WebRTC.
- Wireshark, un analizador de protocolos que permite examinar el intercambio de paquetes, incluyendo consultas a servidores STUN/TURN.

El objetivo de la práctica es realizar una videollamada entre dos dispositivos (como un portátil y un móvil), capturar y analizar los datos generados en el proceso de señalización y transmisión, e interpretar la información relevante, como los códecs utilizados, los servidores STUN/TURN involucrados, la estructura de los paquetes de audio y video, los tiempos de establecimiento de la conexión, y el comportamiento de los candidatos ICE.

Tabla comparativa entre las siguientes aplicaciones:

- meet.jit.si
- talky.io
- whereby.com

Aplicación	Encuesta	Juegos	Compartir Pantalla	Pizarra	Chat
meet.jit.si	X		X	X	X
talky.io		X	X		X
whereby.com			X		X

Tabla 1. Comparación de aplicaciones

Selección **dos aplicaciones de la lista anterior** y probar los siguientes servicios:

- videoconferencia
- multiconferencia
- pantalla compartida
- envío de ficheros

Para las dos aplicaciones elegidas, incluir una breve descripción de cada una de ellas y la lista de servicios que ofrece.

App nº1 (meet.jit.si):

- Descripción
- Lista de servicios
  - Videoconferencia

        - Multiconferecia
        - Compartir pantalla
        - Chat
        - Encuesta
        - Silenciar
        - Cámara
        - Levantar mano
        - Invitar personas
        - Posibilidad de guardar la pizarra como PNG o SVG.
        - Ajustes de rendimiento arriba

App nº2 (whereby.com):

- Descripción
- Lista de servicios
    - Planes de pago diferentes
    - Gratuito
    - Máximo 4 personas

Tabla que refleje los terminales utilizados en la práctica:

(*) Se contempla el uso de los 2 PCs de laboratorio con webcam, y adicionalmente utilizar los smartphone u ordenadores personales.

Terminal	Sistema Operativo	Navegador	Versión Navegador
Terminal 1	Linux	Firefox	2.4.0
Terminal 2	Linux	Firefox	2.4.0
Terminal 3	Android	Google Chrome	134.0.6998.108

Tabla 2. Terminales utilizados en la práctica

(Ordenadores del laboratorio y el móvil de un integrante del equipo)

- **App 1:**

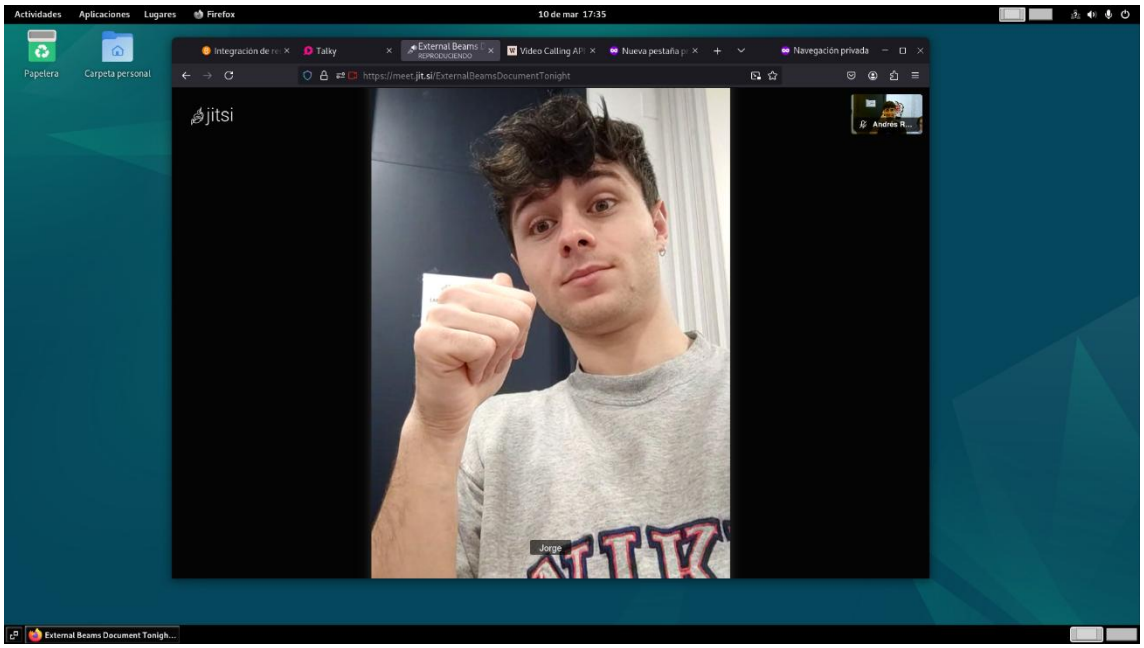


Ilustración 1. Videoconferencia app 1 meet.jit.si

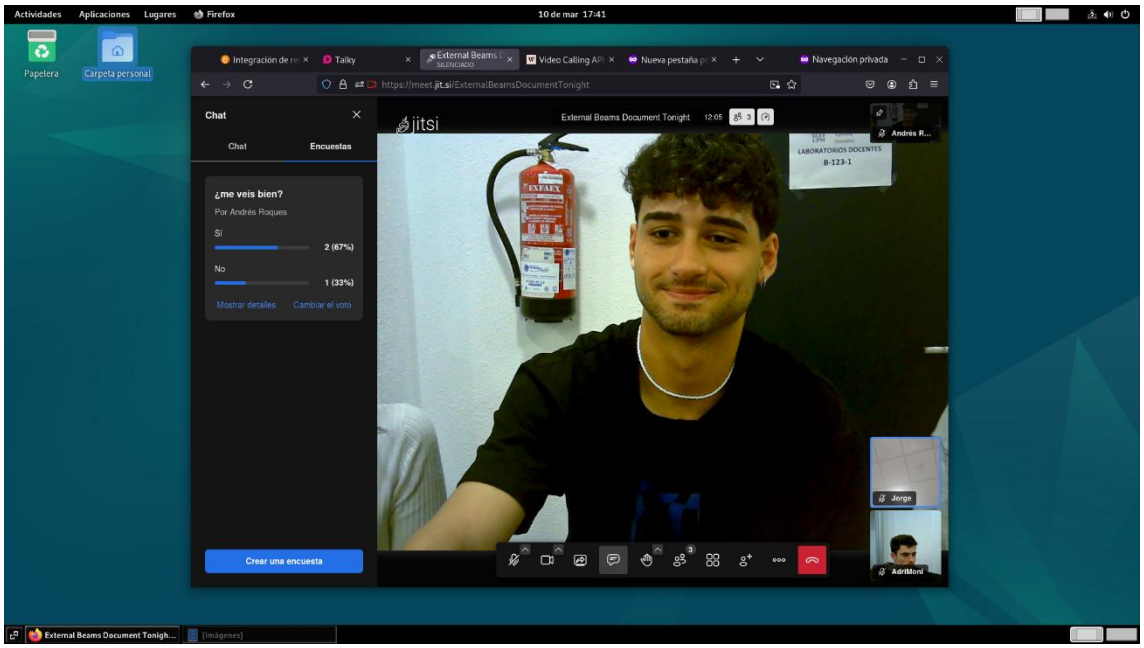


Ilustración 2. Multiconferencia app 1 meet.jit.si y encuesta

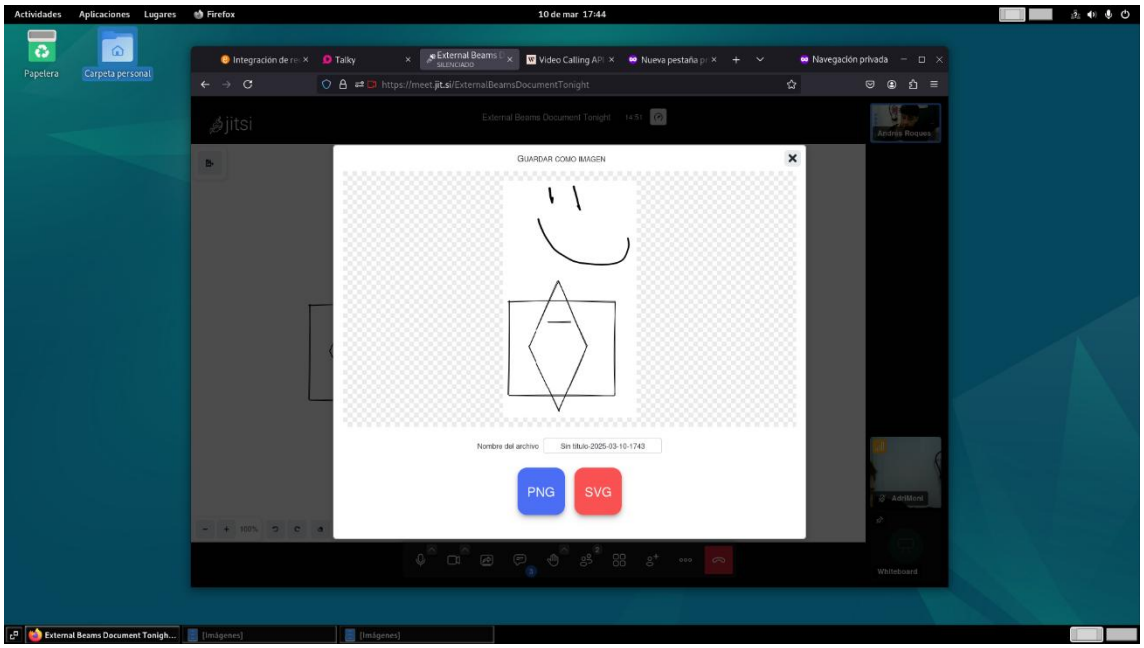


Ilustración 3.Guardar pizarra app 1 meet.jit.si

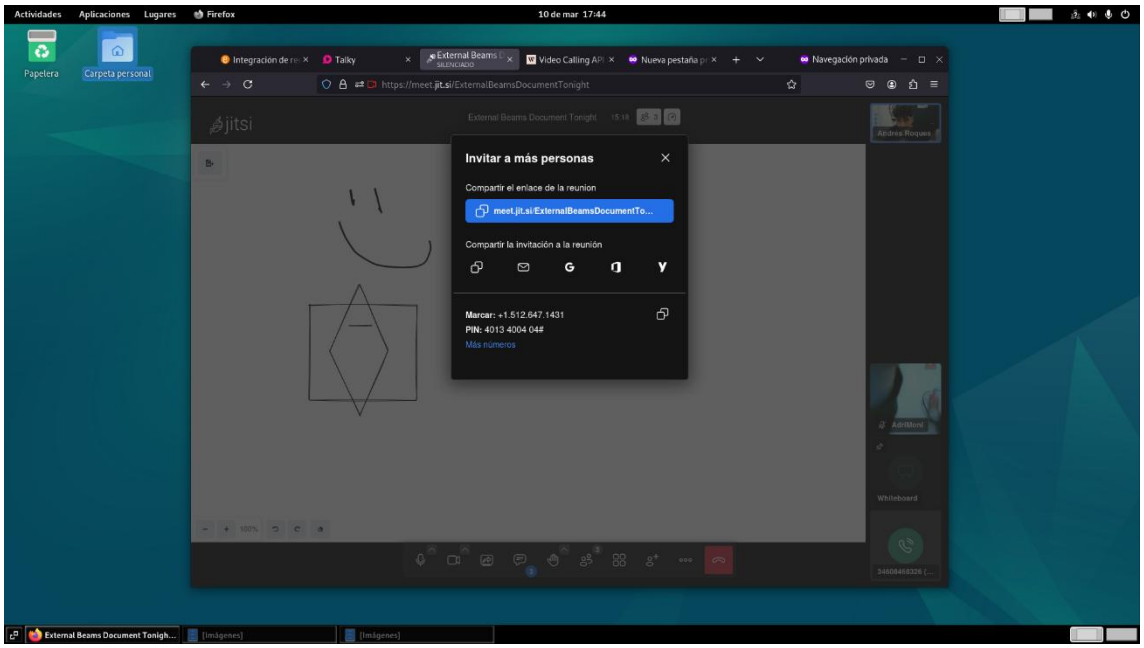


Ilustración 4. Invitar a más personas app 1 meet.jit.si

Captura de pantalla para envío de ficheros app nº1 meet.jit.si:

La aplicación no permite realizar envío de ficheros.

- **App 2:**

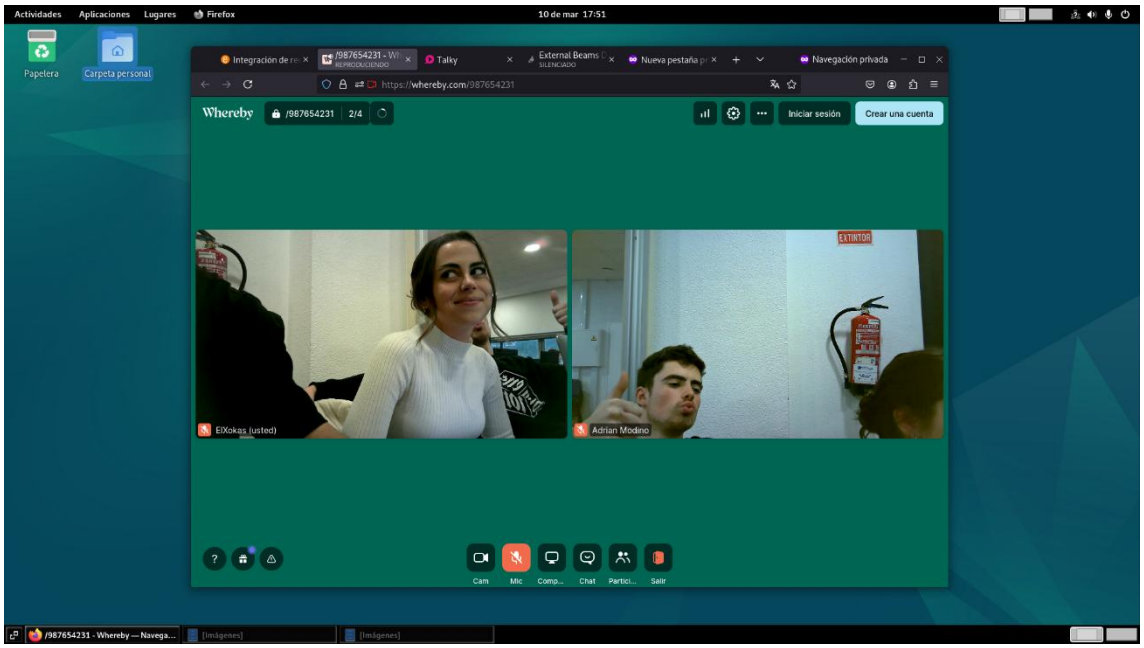


Ilustración 5. Videoconferencia app 2 whereby.com

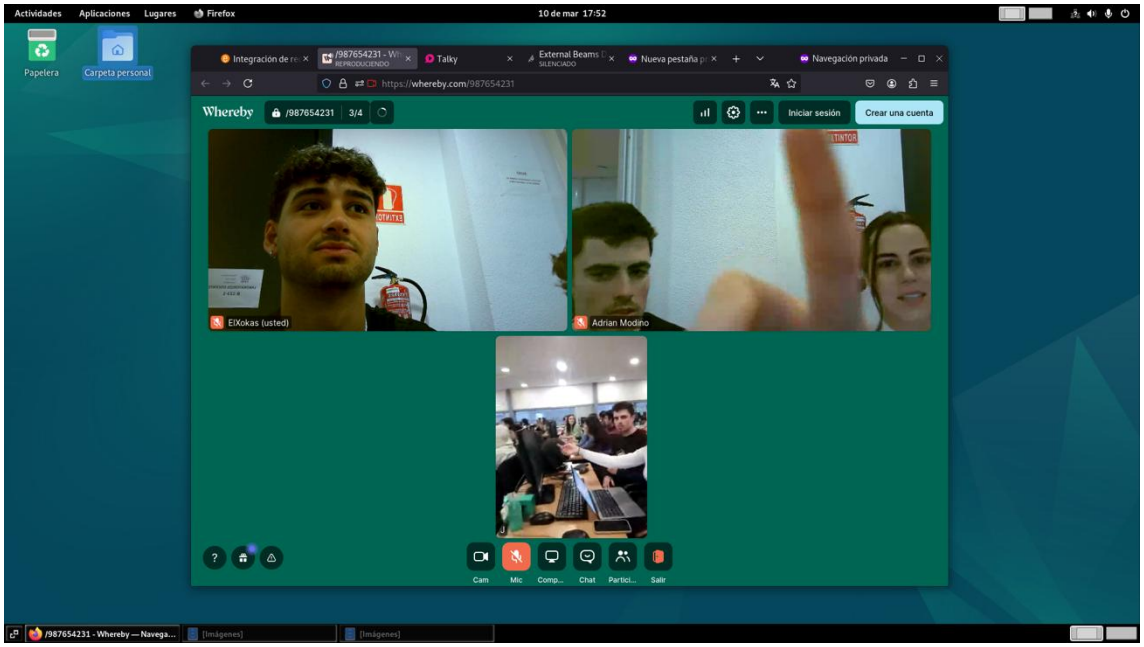


Ilustración 6. Multiconferencia app 2 whereby.com

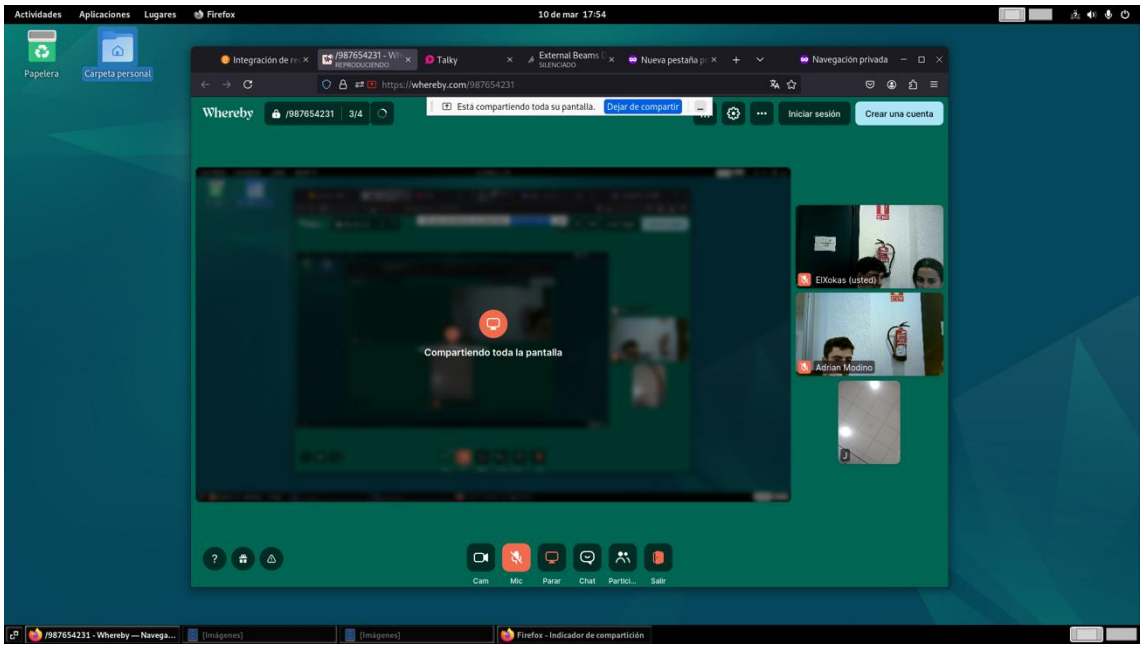


Ilustración 7. Pantalla compartida app 2 whereby.com

Captura de pantalla para envío de ficheros app nº2 whereby.com:

La aplicación no permite realizar envío de ficheros.

Realizar el mismo proceso con una tercera aplicación diferente de las 3 analizadas.

- **App 3 :**

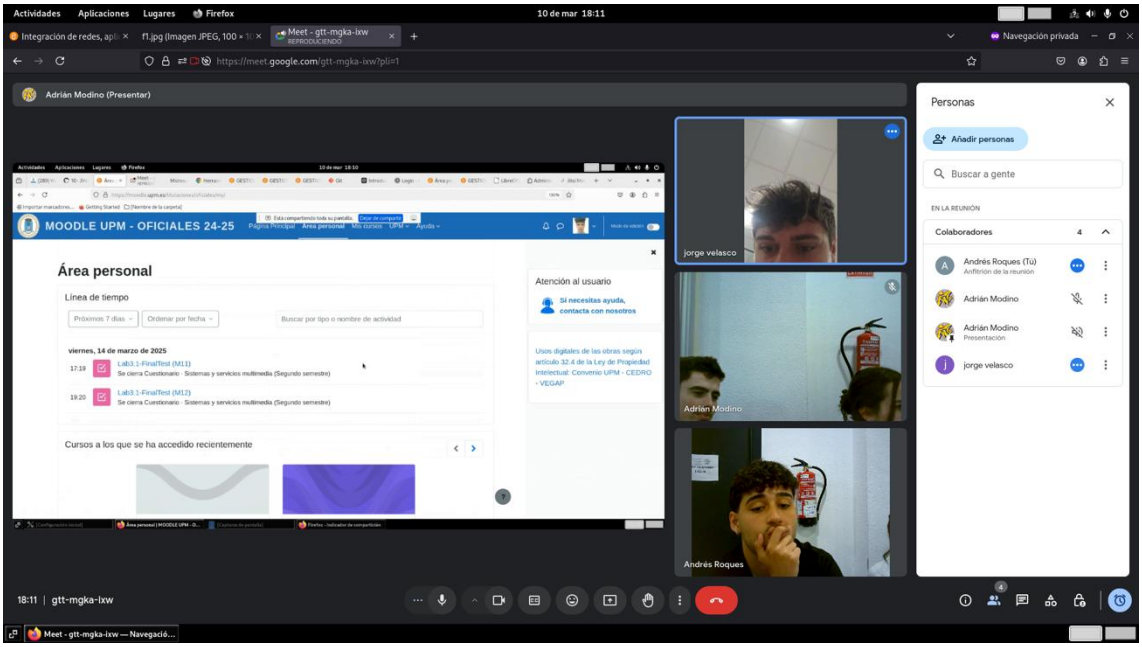


Ilustración 8. Pantalla compartida app 3 meet.google.com

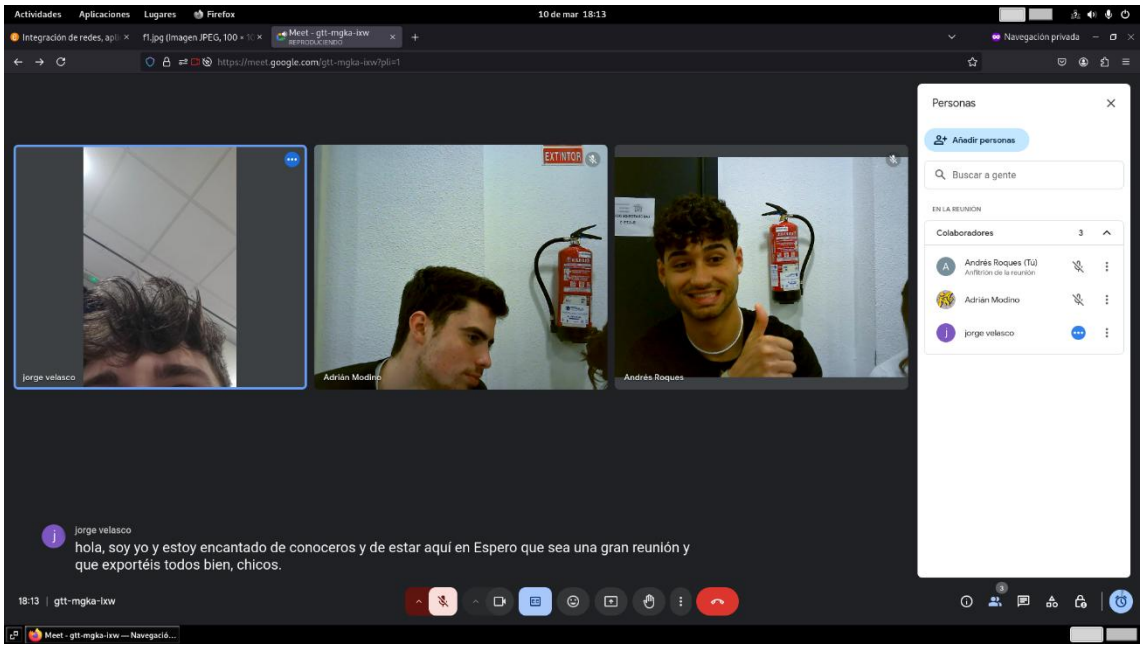


Ilustración 9. Subtítulos app 3 meet.google.com

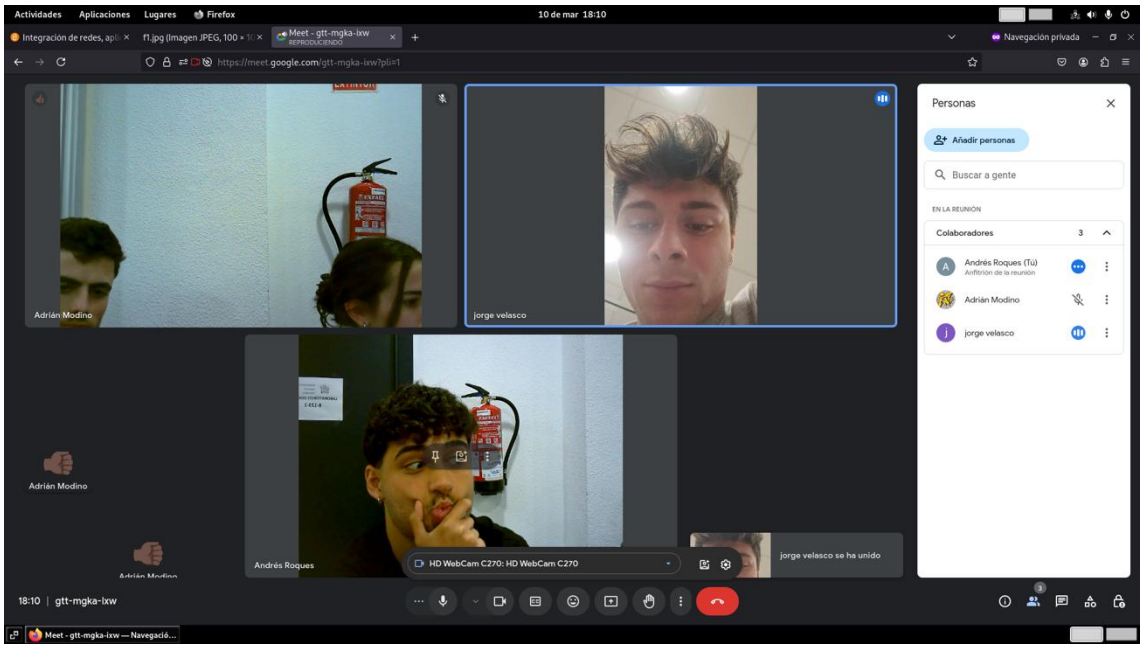


Ilustración 10. Multiconferencia app 3 meet.google.com

Captura de pantalla para envío de ficheros app nº3 meet.google.com:

La aplicación solo permite realizar envío de ficheros como enlace a drive.google.com

Valoración App nº1 (meet.jit.si)

- Flexibilidad y control.
- Interfaz simple.
- Grabación de pantalla de alta calidad.

Valoración App nº2 (whereby.com)

- Simplicidad y facilidad de uso.
- Compartición de pantalla de alta calidad.
- Buena calidad en de video y audio.

Realizar el mismo proceso con una tercera aplicación diferente de las 3 analizadas:

Valoración App nº3 (meet.google.com)

- Más facilidad para la planificación y gestión de reuniones.
- Mejor calidad de video y audio que la anterior.
- Compartición de pantalla de alta calidad.
- Cifrado de extremo a extremo en algunas reuniones.
- Grabación y subtítulos en tiempo real.

3. Análisis Discord

El objetivo de esta parte de la práctica es analizar una videollamada WebRTC con la aplicación https://discord.com/ usando la información de trazado que proporcionan la consola JavaScript y la página de información interna de WebRTC del navegador (chrome://webrtcinternals/ o about:webrtc en el caso de Firefox).

Discord es un programa de comunicación gratuito que permite compartir chats de texto, voz y vídeo con amigos, comunidades de juegos y desarrolladores. Es uno de los servicios más populares para conectarte con otras personas por internet, al contar con cientos de millones de usuarios registrados.

Los servicios que ofrece esta App son:

- Llamadas privadas de audio y video (con opciones activas de compartir pantalla, silenciar micrófono, desactivar video, etc.)
- Mensajes directos
- Creación de servidores para comunicación de varias personas simultáneas.
- Añadir a usuarios a una lista de amigos.

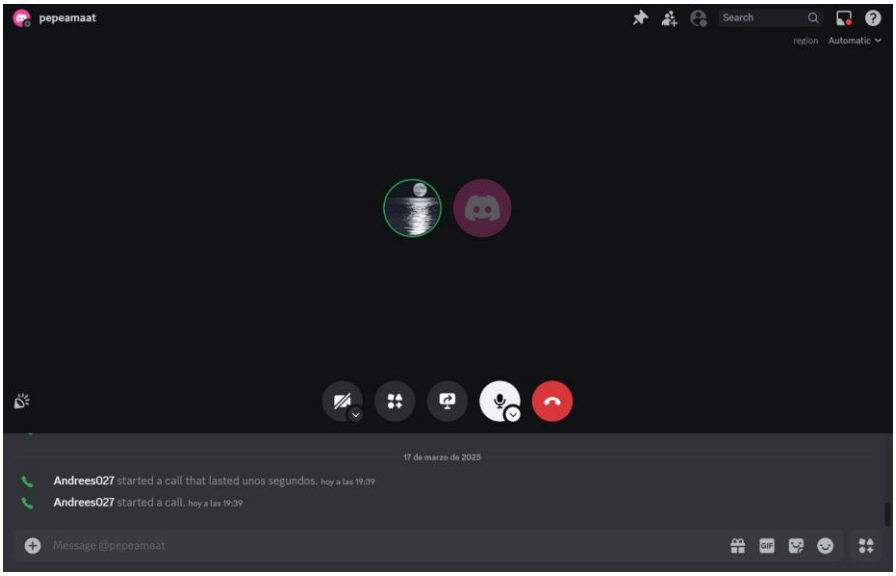


Ilustración 11. Interfaz de usuario de llamada de discord y servicios que ofrece

Em la siguiente figura observamos una captura de como presenta la página about:webrtc el navegador Firefox.

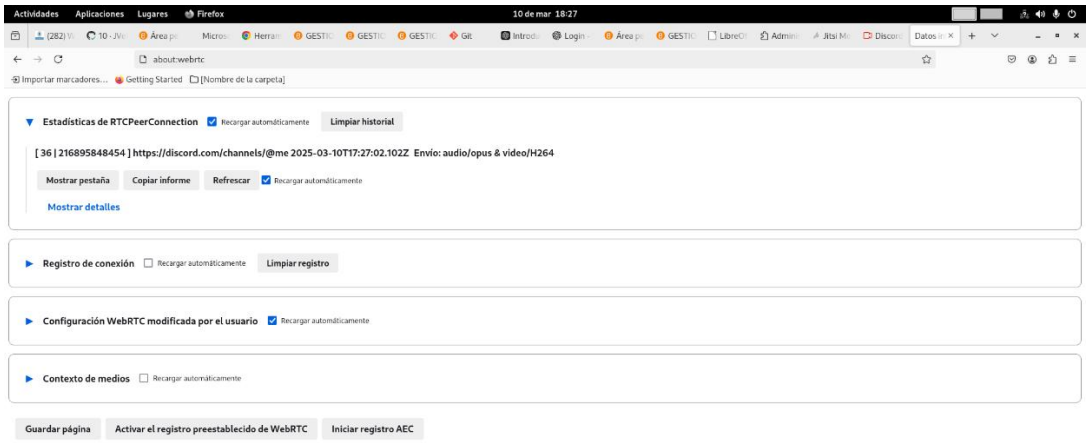


Ilustración 12. Presentación de about:webrtc

Captura de la consola JavaScript durante el establecimiento de la llamada vía Discord.

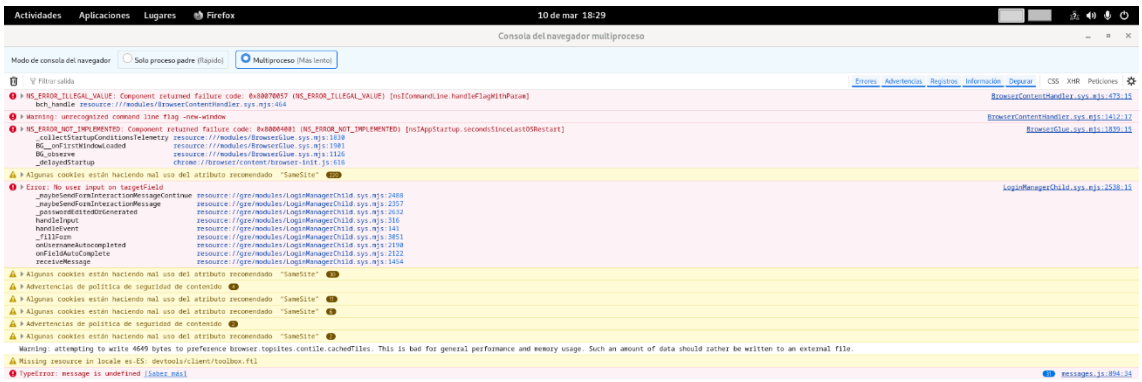


Ilustración 13. Establecimiento de la llamada via discord

Como podemos comprobar en la siguiente captura de pantalla, visualizamos la interfaz de usuario de la aplicación empleada para realizar la videollamada en curso. Comprobamos como solo uno de los dos extremos de la llamada (nuestro usuario registrado) tiene la opción de video activa en la llamada.

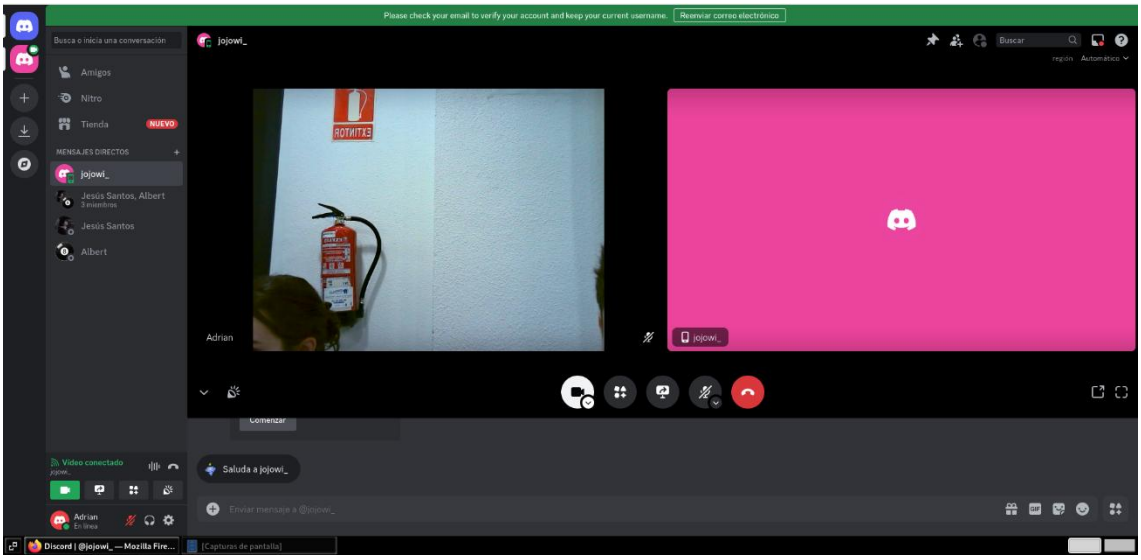


Ilustración 14. Interfaz de usuario en discord durante una llamada

Haciendo uso de Wireshark realizamos una captura para apreciar las direcciones de los servidores STUN empleados. En este caso, vemos en la figura siguiente que las direcciones son 138.4.31.120 y 66.22.241.35.

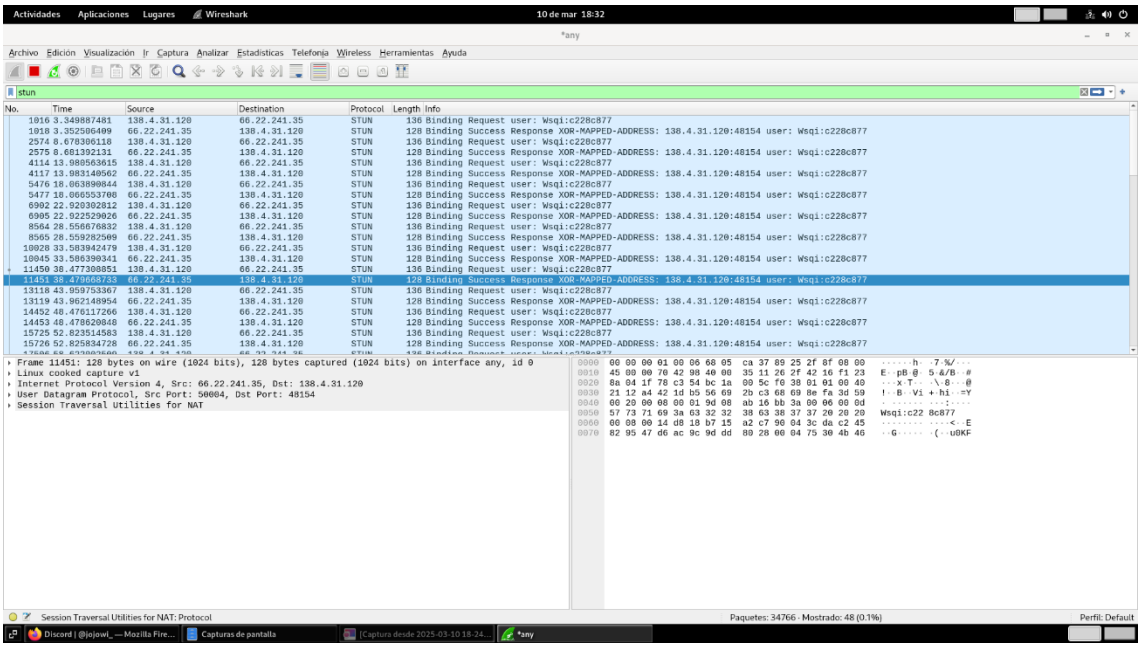


Ilustración 15. Dirección de los servidores STUN empleados

Haciendo uso de about:webrtc visualizamos los códecs que se proponen en la oferta y respuesta SDP durante el establecimiento [Figura XX], las torres de protocolos usados para os paquetes de audio y video [Figuras XX y XX], los candidatos ICE que envía cada lado de la conexión [Figura XX] y las gráficas sobre prestaciones de los flujos de audio y vídeo [Figura XX].

SDP AUDIO

REMOTO: m=audio 50004 UDP/TLS/RTP/SAVPF

Información SDP REMOTO:

- **m=** → Indica el inicio de una descripción de medio en SDP.
- **audio** → Es el tipo de medio que se está negociando. En este caso, **audio**.
- **50004** → Es el puerto en el que se recibirá el audio RTP.
- **UDP/TLS/RTP/SAVPF** → Especifica el protocolo de transporte:
  - **UDP** → Usa el protocolo UDP.
  - **TLS** → Emplea la prestación de cifrado TLS.
  - **RTP** (Real-time Transport Protocol) → Transmisión instantánea del audio.
  - **SAVPF** (Secure Audio Video Profile with Feedback) → Perfil de RTP que soporta cifrado con SRTP y retroalimentación RTCP.
- **109** → Es el Payload Type (PT) que se usará en la transmisión RTP. Se refiere a un códec específico, que como comprobaremos acto seguido, se trata de Opus.

LOCAL (oferta): m=audio 9 UDP/TLS/RTP/SAVPF 109 9 0 8 101

Información SDP LOCAL:

El puerto RTP de valor 9, es un placeholder especial en WebRTC y VoIP que indica que el puerto real se negociará dinámicamente. Nos indica que usa los mismos protocolos de transporte que los empleados por el REMOTO, y por último, vemos que los códecs soportados son aquellos con Payload Types: 109, 9, 0, 8 y 101.

En este caso, como el único códec en común en ambos extremos es Opus (PT=109), será el empleado.

Códec de audio empleado: Código 109: Opus.

a=rtpmap:109 opus/48000/2

Esta línea de información se entiende de la siguiente manera:

- **a=** → Indica un atributo en SDP.
- **rtpmap**: → Define un mapeo entre un número de payload dinámico y un códec.
- **109** → Es el Payload Type (PT), un identificador numérico que se usa en RTP para indicar el códec específico que se está utilizando. Este número es dinámico y puede cambiar en cada sesión.
- **opus** → Es el nombre del códec. En este caso, Opus, un códec de audio de alta calidad y baja latencia, muy usado en WebRTC y VoIP.
- **48000** → Es la frecuencia de muestreo en Hz. Opus soporta múltiples frecuencias, pero aquí se especifica 48 kHz.
- **2** → Indica el número de canales de audio. En este caso, 2 significa que es estéreo.

SDP VIDEO

```
REMOTO:  m=video 50004 UDP/TLS/RTP/SAVPF 126 127
```

Siguiendo la línea del análisis que hemos visto anteriormente, esta línea nos da información SDP sobre medio de vídeo, alojado en el puerto 50004. Tiene una torre de protocolos de trasporte del tipo: UDP/TLS/RTP/SAVPF, al igual que en audio, y con dos códecs aceptados, cuyos códigos son 126 y 127.

```
LOCAL:  m=video 0 UDP/TLS/RTP/SAVPF 126 127 120 124 121 125 97 98 123 122 119
```

En el caso de la oferta, vemos que el puerto RTP es 0 lo que indica que esta línea está deshabilitada. Esto puede significar que el lado que genera este SDP no desea enviar o recibir video. Además vemos que los PT soportados son: 126, 127, 120, 124, 121, 125, 97, 98, 123, 122 y 119.

```
a=rtpmap:126 H264/90000
a=rtpmap:127 rtx/90000
```

Analizamos los códecs empleados y vemos

Códec de código 126 → H.264 con frecuencia de muestreo de 90kHz.
Códec de código 127 → rtx con frecuencia de muestreo de 90kHz.

La frecuencia de muestreo para archivos de video siempre suele ser 90kHz (vemos que ambos casos lo cumplen) ya que facilita el manejo del tiempo en RTP.

RTX (Retransmisión) es un mecanismo para reenviar paquetes RTP perdidos, mejorando la calidad en redes con pérdidas de paquetes.

Este SDP indica que:

- H.264 (PT 126) se usará como el códec de video principal.
- RTX (PT 127) permitirá retransmitir paquetes perdidos, mejorando la confiabilidad del video.

Esto lo comprobamos en la siguiente línea de información SDP donde comprobamos que ambos códecs están asociados de la forma que RTX (PT 127) se usa para retransmitir paquetes de H.264 (PT 126).

```
a=fmtp:127 apt=126
```

Tiempo de establecimiento

Analizando la ilustración siguiente, donde nos ayudamos de la pestaña “red” de la consola del navegador. Observaos que indica un tiempo de 114 ms de espera entre la llamada y el establecimiento de la misma.

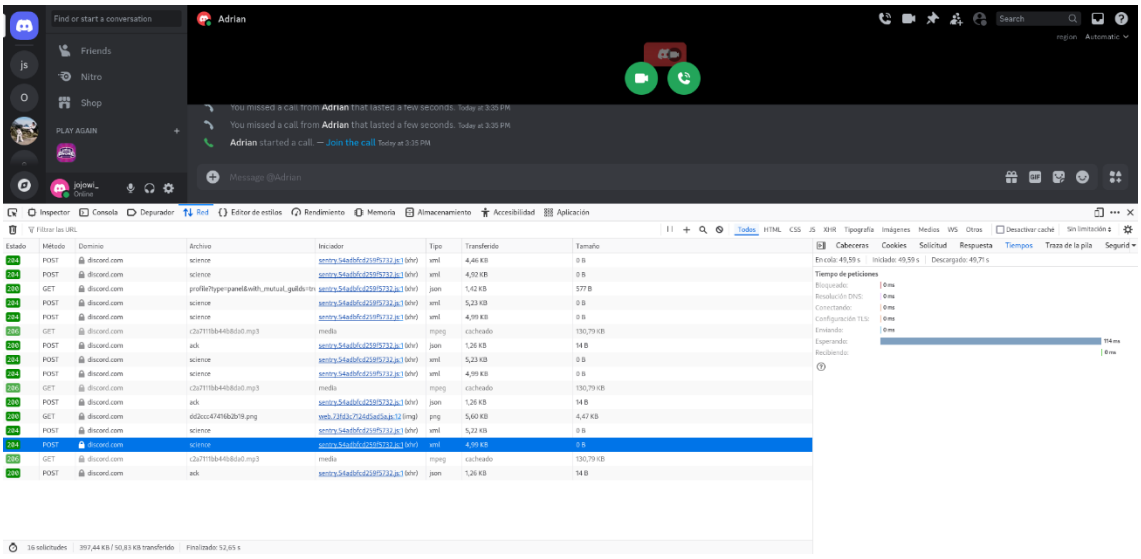


Ilustración 16. Tiempo de establecimiento de llamada

En la siguiente ilustración comprobamos las Estadísticas Interactive Connectivity Establishment (ICE). Nos indica que la conexión ha tenido éxito y se ha establecido la comunicación (estado succeeded), indica el candidato elegido como el mejor para la conexión (nominated true) y que este aún está en uso (selected true).

Candidato local: *186d78ce-4cfe-4d7d-bf67-9715cc17b33f.local:41371/udp(host) [non-proxied]*. Direccion generada localmente, con Puerto 41371 y protocolo UDP, sin pasar por un proxy (indicado por [non proxied]).

Candidato remoto: *66.22.241.166:50004/udp(host)*. Dirección IP pública del otro participante en la llamada. Usa el puerto 50004, protocolo UDP y (host) indica que es una IP directa, sin necesidad de TURN.

Estadísticas ICE

Candidatos entrantes (llegando tras la respuesta) son destacados en azul

Estado ICE	Nominado	Seleccionado	Candidato local	Candidato remoto	ID del componente	Prioridad	Bytes enviados:	Bytes recibidos:
succeeded	true	true	186d78ce-4cfe-4dd7-bf67-9715cc17b33f.local:41371/udp(host) [non-proxied]	66.22.241.166:50004/udp(host)	1	9115052702656920 00	157224129	149251859

Reinicios de ICE: 0
Vueltas atrás de ICE: 0

Todos los candidatos no procesados

Ocultar candidatos no procesados

Candidato local no procesados
host(IP4:138.4.31.86:41371/UDP)
host(IP4:138.4.31.86:60757/TCP) active

Candidato remoto no procesados
candidate:1 1 UDP 4261412862 66.22.241.166 50004 typ host

Ilustración 17. Estadisticas ICE

En las siguientes gráficas de la videollamada, Ilustración 18, nos encontramos información acerca de los siguientes parámetros de audio y video:

- Frames por segundo (fps) en video:

El promedio es en torno a 20 fps con pequeñas fluctuaciones, lo cual es bajo para una transmisión fluida (idealmente 30-60 fps). Esto podría indicar problemas de ancho de banda o procesamiento.

- Pérdida de paquetes (packetsLost)

Se observan pérdidas de paquetes en audio y video, aunque en valores bajos, negativos y positivos. Esto nos da la idea de que pueden ser anomalía en el cálculo de las estadísticas. Centrándonos en la media que es en torno a 0, nos sugiere que la videollamada se produce correctamente sin interrupciones o distorsiones.

- Jitter (variabilidad en el retraso de paquetes)

Los valores de jitter están generalmente bajos, lo que significa buena estabilidad en la red.

- Bytes enviados y recibidos (bytesSent, bytesReceived)

La cantidad de datos transmitidos parece estable. Se observa una tasa de kilobits por segundo (kbps) en video y audio, con una transmisión de audio en estéreo.

- Round Trip Time (RTT) - Latencia ida y vuelta.

RTT está en 4 ms, lo cual es muy bueno, indicando baja latencia.

- Retransmisión y corrección de errores (rtx, FEC)

Se observa un nackCount (Número de retransmisiones solicitadas). Uso de RTX (Retransmisión de paquetes perdidos) y ULPFEC (Corrección de errores adelantada).

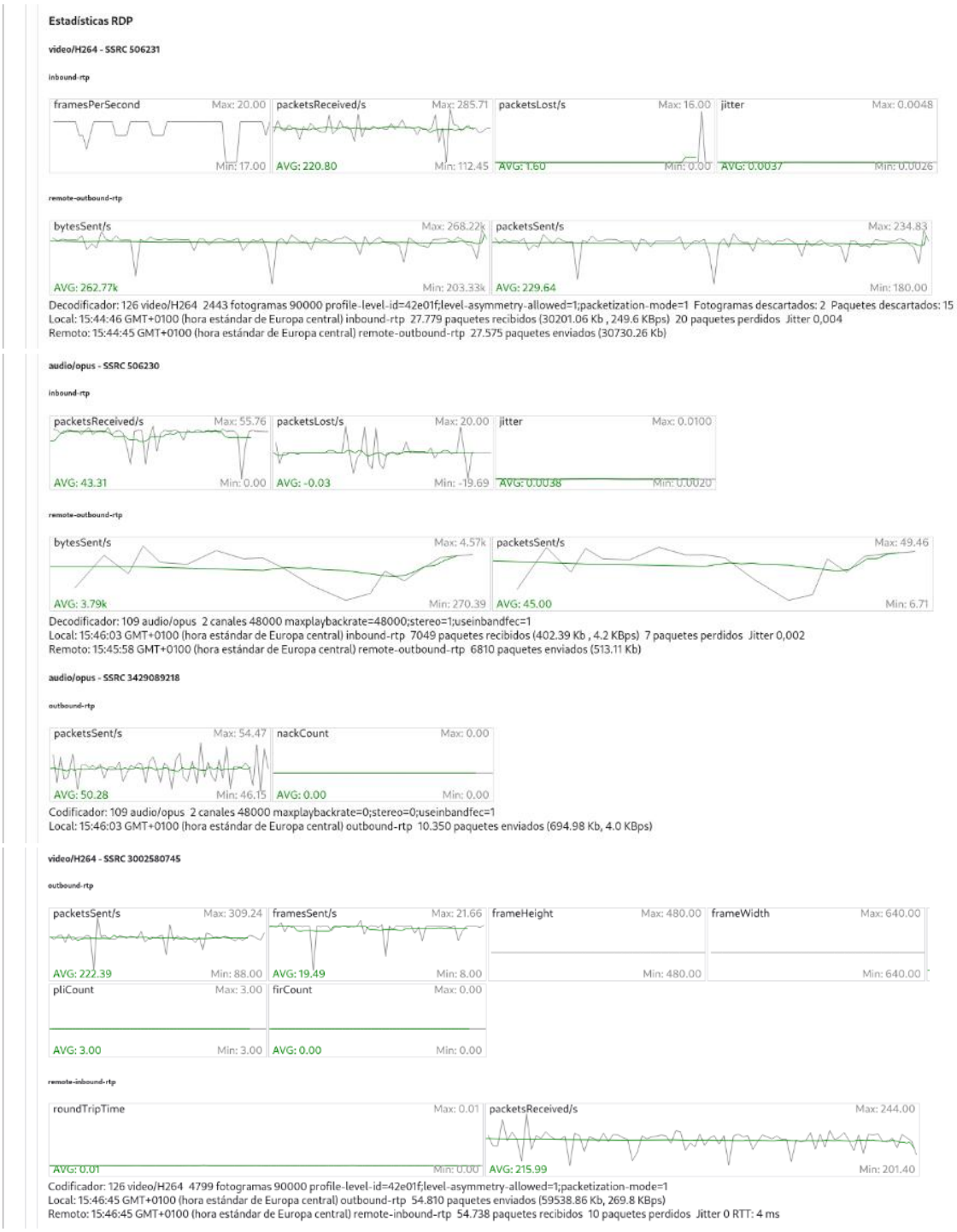


Ilustración 18. Gráficas de videollamada

4. Despliegue de una aplicación WebRTC

En esta sección se va a montar una aplicación WebRTC sencilla a partir de componentes ya desarrollados. Los ficheros HTML5 y JavaScript necesarios fueron proporcionados en Moodle de la asignatura. Se van a usar herramientas Node.js, socket.io y node-static además de un contenedor Docker.
En primer lugar se van a probar todos los componentes de la aplicación individualmente para a continuación combinarlos en una única aplicación y finalmente usarla para realizar una videollamada con un usuario local y otro remoto.
Toda la sección se ha realizado en los ordenadores del laboratorio.

Pruebas de captura y visualización:

index.html

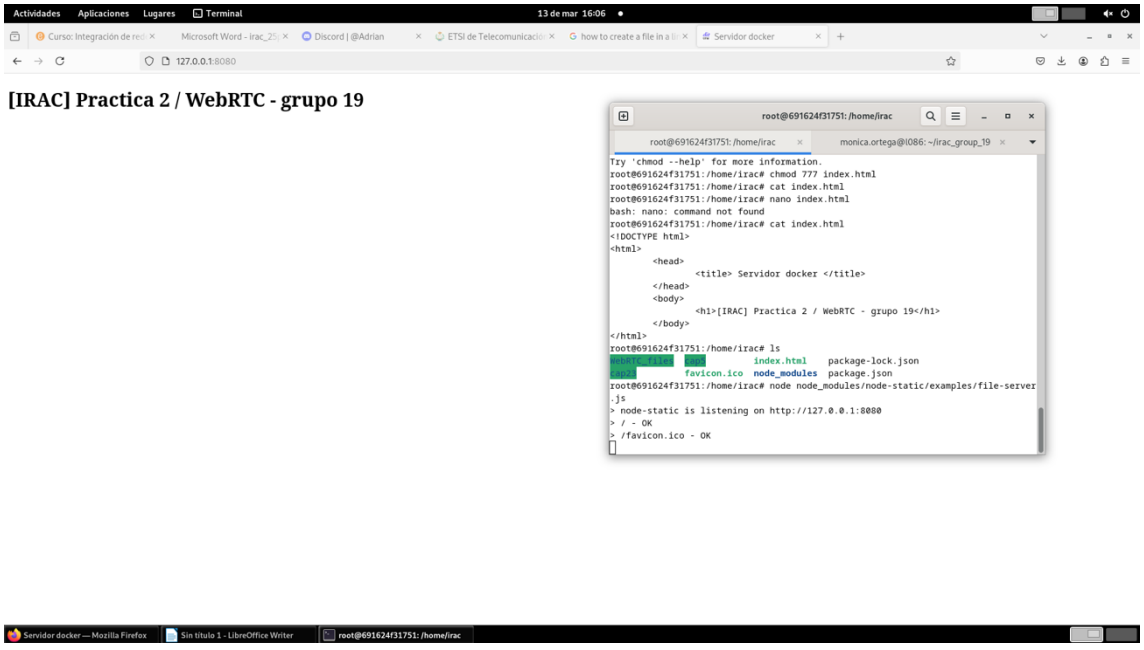


Ilustración 19. Index.html

En esta primera imagen podemos observar la interfaz básica del servidor web desplegado dentro del contenedor Docker.

En primer lugar, se puede observar en la ventana de comandos la estructura del fichero index.html. Su estructura es muy sencilla, únicamente modifica el título de la página a ‘Servidor docker’ y muestra un breve mensaje en el cuerpo que indica el número de grupo 19.

En la misma ventana de comandos a continuación se observa como con el comando “node node_modules/node-static/examples/file-server.js” se lanza el servidor web, dentro del contenedor Docker, usando la herramienta node-static.

En este punto, se puede acceder a la página index.html en la dirección http://localhost:8080, como se puede observar en la imagen.

getUserMedia.html

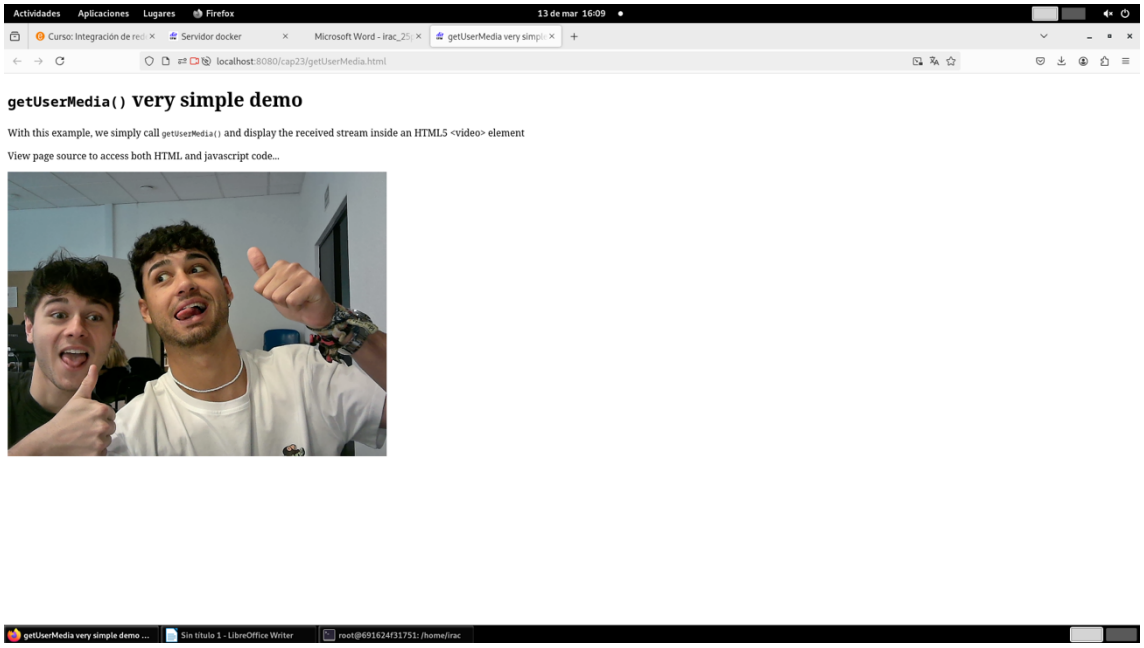


Ilustración 20. getUserMedia.html

Tras lanzar el servidor web descrito anteriormente se puede acceder a la página getUserMedia.html, en la que, tras dar permisos de acceso al navegador, se muestra el video capturado por la cámara del ordenador.

LocalPeerConnection.html

La página LocalPeerConnection.html ilustra el uso del API RTCPeerConnection. El fichero HTML muestra un interfaz con dos ventanas de vídeo como el que se usaría para una videollamada básica.

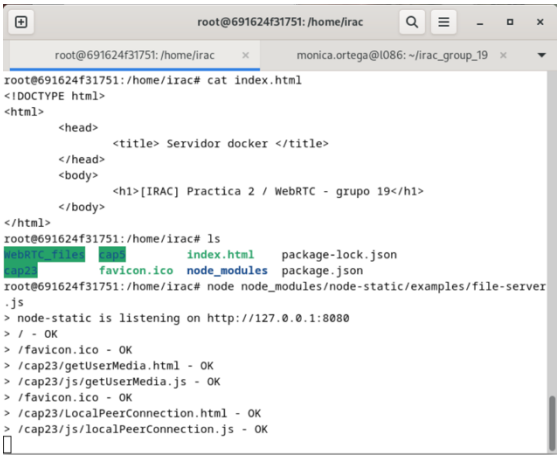


Ilustración 21. LocalPeerConnection en la consola de comandos

En la siguiente imagen se muestra la interfaz de la página inicial. Una ventana representa el video local y la otra ventana muestra el video remoto.



Ilustración 22. LocalPeerConnection.html, Página de inicio

Tras acceder a 'Start' se inicia el video local capturado por getUserMedia() desde la cámara.

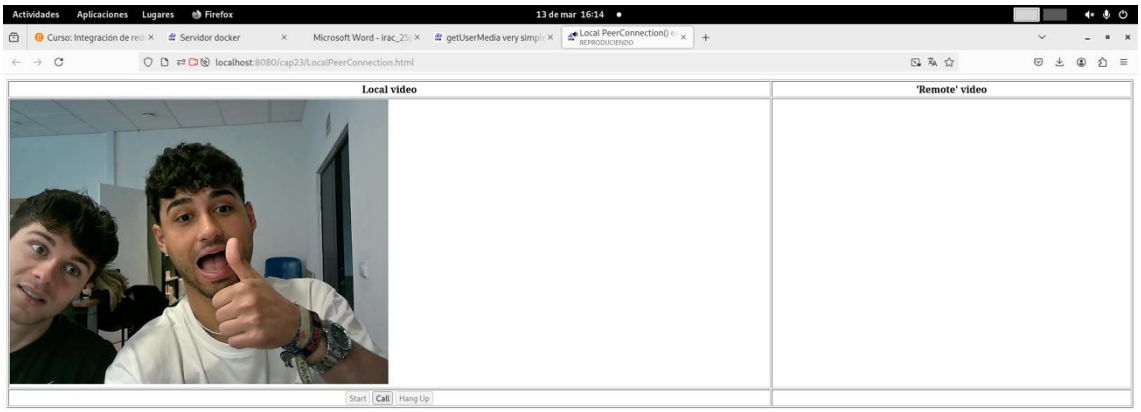


Ilustración 23.LocalPeerConnection.html, Start

A continuación, presionando el botón 'Call' se muestra el video remoto, que en esta prueba es una copia del video local. Esto simula la conexión entre dos usuarios, pero en este caso, ambos están en la misma máquina.

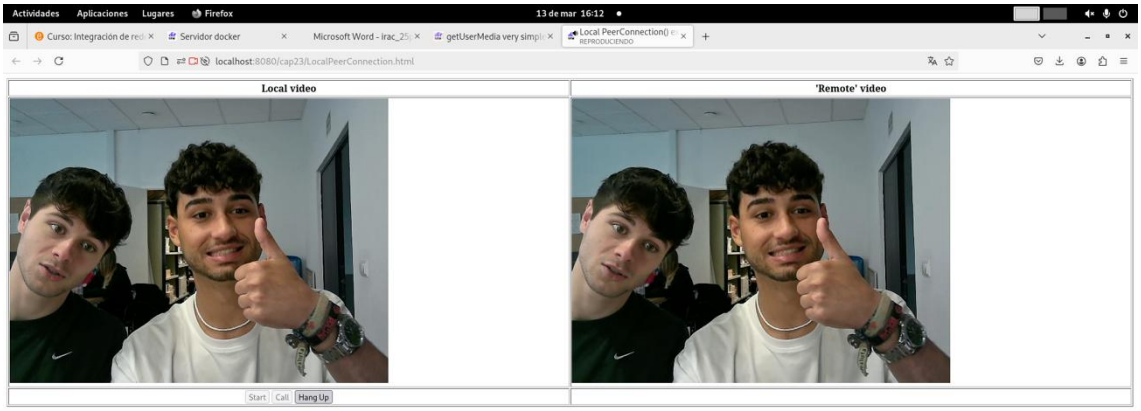


Ilustración 24. LocalPeerConnection.html, Call

Finalmente, la página ofrece la opción de 'Hang Up' que corta la comunicación con el servidor remoto y se deja de recibir la señal de su cámara.

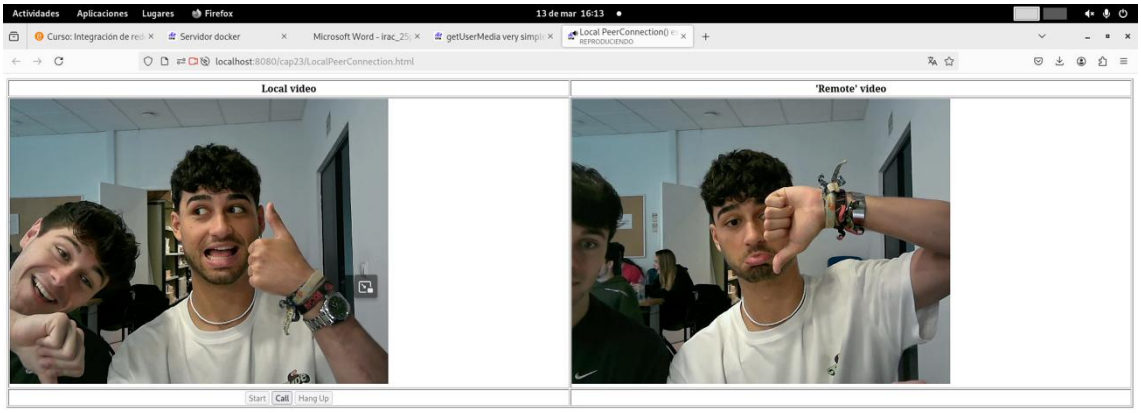


Ilustración 25. LocalPeerConnection.html. Hang Up

dataChannel.html

La página dataChannel.html utiliza el API de datos Peer-to-Peer DataChannel. El fichero HTML muestra un interfaz que permite enviar mensajes de texto.

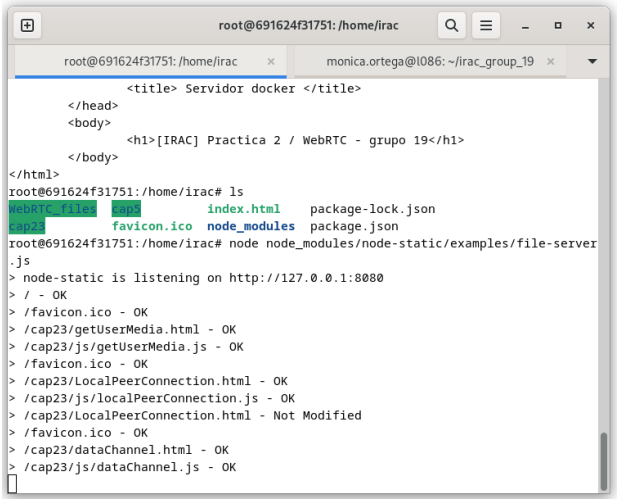


Ilustración 26. dataChannel en la consola de comandos

En la página se pueden observar dos cuadros de texto para escribir y recibir los mensajes y tres botones con las opciones ‘Start’ para iniciar la conexión, ‘Send’ para enviar el mensaje de texto y ‘Stop’ para cortar la conexión.

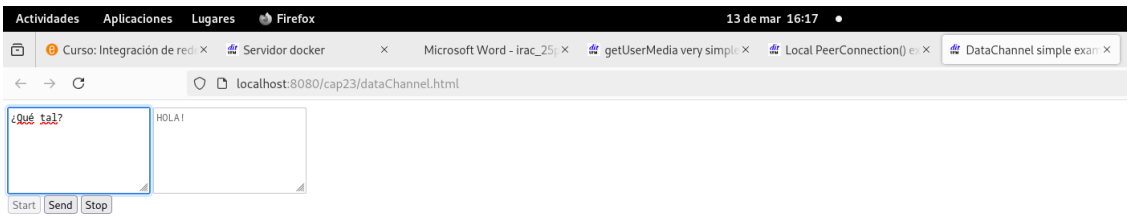


Ilustración 27. dataChannel.html

Pruebas de videollamada:

En esta sección se van a integrar todos los elementos anteriores para crear una aplicación WebRTC que permita hacer videollamadas y enviar mensajes entre dos navegadores.

El nuevo servidor se lanza con el comando ‘node cap5/completeNodeServerWithDataChannel.js’ y es accesible en la dirección ‘https://localhost:8080/cap5/'

En primer lugar, para comprobar su funcionamiento, se ha accedido a la página desde dos ventanas del mismo equipo y se ha dado el mismo nombre a la comunicación en ambos.

En este caso el video local y remoto coinciden ya que ambos están conectados desde el mimo equipo.

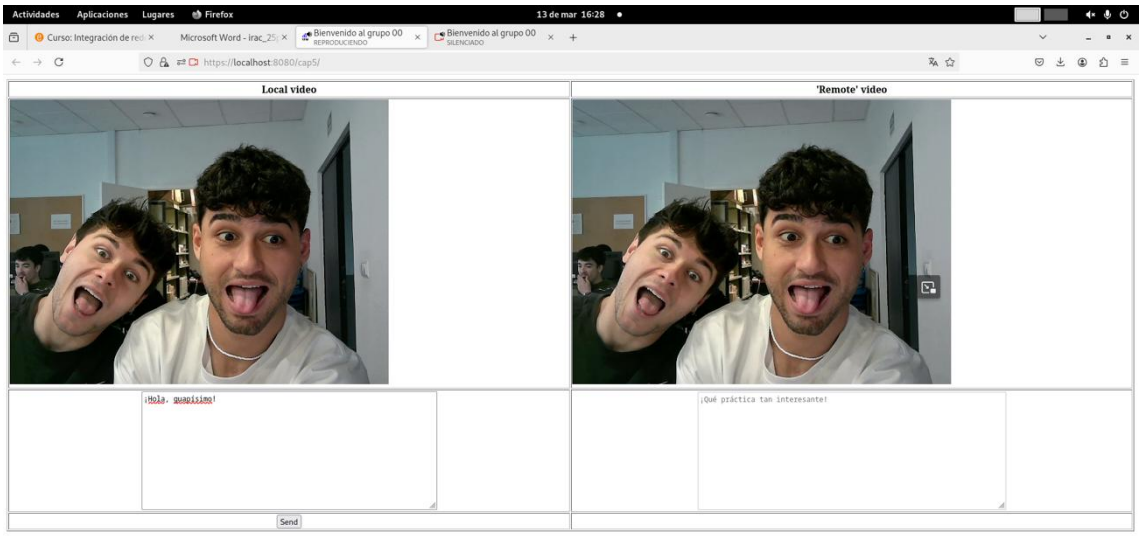


Ilustración 28. Página /cap5/ en un equipo

En la imagen se puede observar la sala creada, con el título editado, el correcto funcionamiento de los dos videos y del canal de datos.

Finalmente, se probó el servicio en remoto utilizando los dos PCs de laboratorio conectados a la misma red. Para cargar la página desde el nuevo ordenador se debe acceder con la dirección IP del ordenador donde se ha ejecutado el servidor.

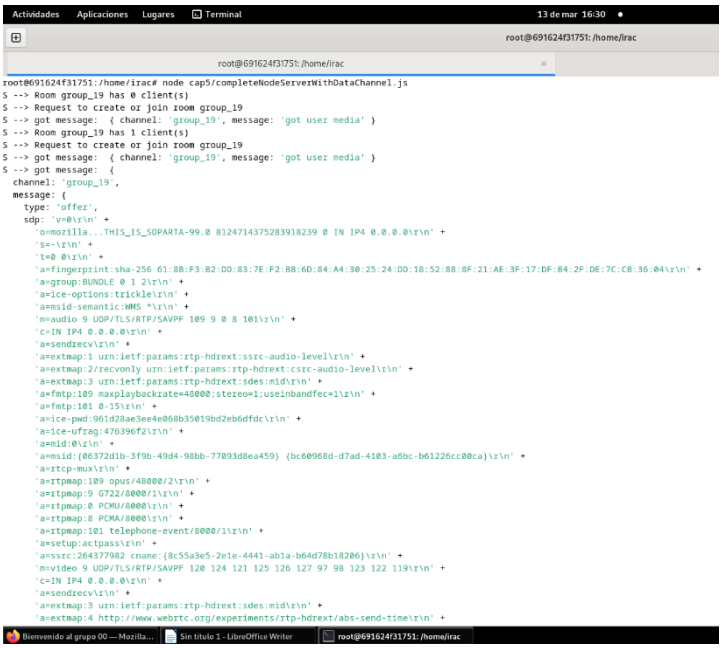


Ilustración 29. SDP en el terminal

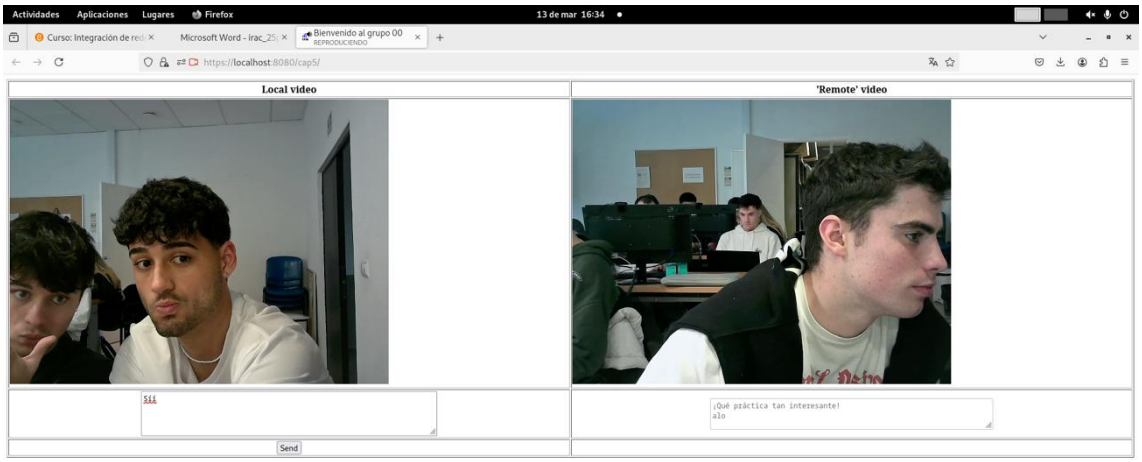


Ilustración 30. Página /cap5/ en remoto

Candidatos ICE intercambiados

En WebRTC, ICE (Interactive Connectivity Establishment) utiliza diferentes tipos de candidatos para establecer la mejor conexión posible entre dos dispositivos.

Los candidatos seleccionados son los siguientes:

- Candidato local: 38.4.31.86:57341/udp (host)
  Tipo: host, dirección IP local del dispositivo dentro de la red LAN
- Candidato remoto: 138.4.31.85:60773/udp (prflx)
  Tipo: prflx, dirección pública detectada mediante el NAT en la comunicación directa entre pares.



Estadísticas ICE

Candidatos entrantes (llegando tras la respuesta) son destacados en azul

Estado ICE	Nominado	Seleccionado	Candidato local	Candidato remoto	ID del componente	Prioridad	Bytes enviados:	Bytes recibidos:
succeeded	true	true	138.4.31.86:57341/udp(host) [non-proxied]	138.4.31.85:60773/udp(prflx)	1	7962083765675360000	47715803	49021106

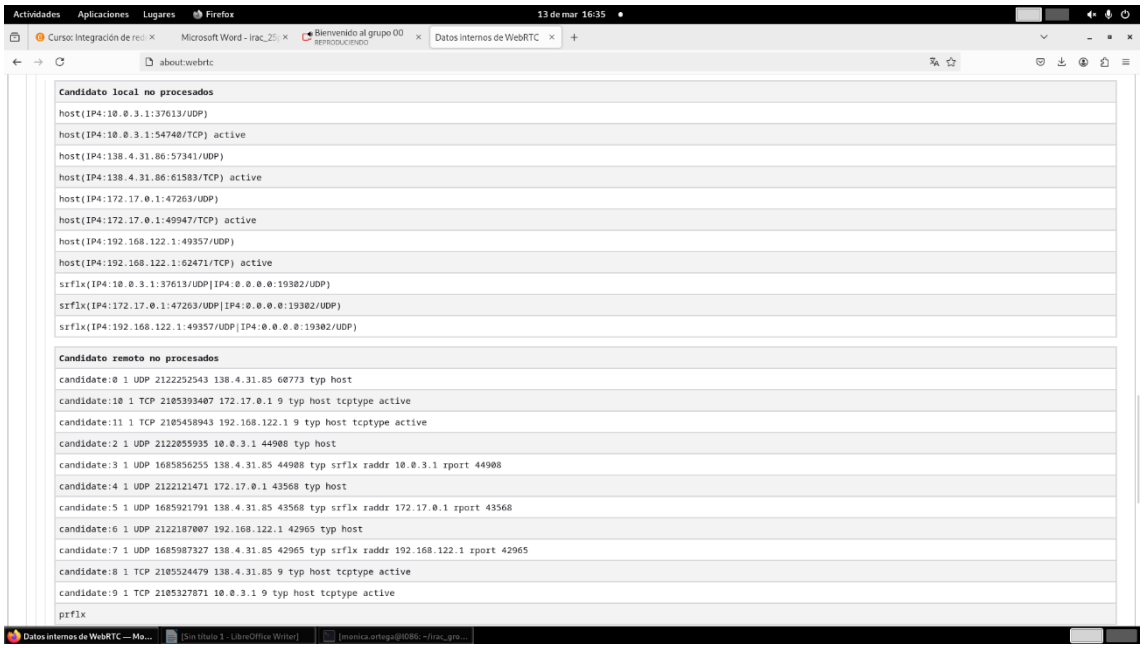
Reinicios de ICE: 0
Vueltas atrás de ICE: 0

Todos los candidatos no procesados

Mostrar candidatos no procesados

Ilustración 31. Cap5. Candidatos ICE seleccionados

En la siguiente imagen se presentan todos los candidatos ICE no procesados:



Actividades Aplicaciones Lugares Firefox 13 de mar 16:35

about:webrtc

Candidato local no procesados

```
host(IP4:10.0.3.1:37613/UDP)
host(IP4:10.0.3.1:54740/TCP) active
host(IP4:138.4.31.86:57341/UDP)
host(IP4:138.4.31.86:61583/TCP) active
host(IP4:172.17.0.1:47263/UDP)
host(IP4:172.17.0.1:49947/TCP) active
host(IP4:192.168.122.1:49357/UDP)
host(IP4:192.168.122.1:62471/TCP) active
srflx(IP4:10.0.3.1:37613/UDP|IP4:0.0.0.0:19302/UDP)
srflx(IP4:172.17.0.1:47263/UDP|IP4:0.0.0.0:19302/UDP)
srflx(IP4:192.168.122.1:49357/UDP|IP4:0.0.0.0:19302/UDP)
```

Candidato remoto no procesados

```
candidate:0 1 UDP 2122252543 138.4.31.85 60773 typ host
candidate:10 1 TCP 2105393407 172.17.0.1 9 typ host tcptype active
candidate:11 1 TCP 2105458943 192.168.122.1 9 typ host tcptype active
candidate:2 1 UDP 2122055935 10.0.3.1 44908 typ host
candidate:3 1 UDP 1685856255 138.4.31.85 44908 typ srflx raddr 10.0.3.1 rport 44908
candidate:4 1 UDP 2122121471 172.17.0.1 43568 typ host
candidate:5 1 UDP 1685921791 138.4.31.85 43568 typ srflx raddr 172.17.0.1 rport 43568
candidate:6 1 UDP 2122187007 192.168.122.1 42965 typ host
candidate:7 1 UDP 1685987327 138.4.31.85 42965 typ srflx raddr 192.168.122.1 rport 42965
candidate:8 1 TCP 2105524479 138.4.31.85 9 typ host tcptype active
candidate:9 1 TCP 2105327871 10.0.3.1 9 typ host tcptype active
prflx
```

Ilustración 32. Cap5. Candidatos ICE no procesados

SDP

Como se vio anteriormente SDP es el protocolo que describe los parámetros multimedia para establecer una sesión en WebRTC. A continuación se muestra la principal información extraída del SDP local y del SDP remoto, que en este caso toda la información coincide entre ambos.

SDP AUDIO

```
m=audio 9 UDP/TLS/RTP/SAVPF 109 9 0 8 101
```

Protocolos usados:

- Transporte: UDP
- Prestación: TLS
- Aplicación: RTP y SAVPF

Códecs de audio disponibles: Códigos 109, 9, 0, 8, 101

```
a=rtpmap:109 opus/48000/2
a=rtpmap:9 G722/8000/1
a=rtpmap:0 PCMU/8000
a=rtpmap:8 PCMA/8000
a=rtpmap:101 telephone-event/8000/1
```

- Códec 109: Opus
- Códec 9: G.722
- Códec 0: PCMU
- Códec 8: PCMA
- Códec 101: telephone-event

SDP VIDEO

```
m=video 9 UDP/TLS/RTP/SAVPF 120 124 121 125 126 127 97 98 123 122 119
```

Protocolos usados:

- Transporte: UDP
- Prestación: TLS
- Aplicación: RTP y SAVPF

Códecs de video disponibles: Códigos 120 124 121 125 126 127 97 98 123 122

```
a=rtpmap:120 VP8/90000
a=rtpmap:124 rtx/90000
a=rtpmap:121 VP9/90000
a=rtpmap:125 rtx/90000
a=rtpmap:126 H264/90000
a=rtpmap:127 rtx/90000
a=rtpmap:97 H264/90000
a=rtpmap:98 rtx/90000
a=rtpmap:123 ulpfec/90000
a=rtpmap:122 red/90000
a=rtpmap:119 rtx/90000
```

- Códec 120: VP8
- Códec 124, 125, 127, 98, 119: RTX
- Códec 121: VP9
- Códec 126, 97: H.264
- Códec 123: ULPFEC
- Códec 122: RED

SDP DATOS

```
m=application 9 UDP/DTLS/SCTP webrtc-datachannel
```

Protocolos usados:

- Transporte: UDP y SCTP
- Prestación: DTLS

```
a=sctp-port:5000
a=max-message-size:1073741823
```

También se indica el puerto donde se transmiten los mensajes (5000) y su tamaño máximo

Estadísticos de la videollamada

El parámetro audio/opus - SSRC 8304448 representa el flujo audio de entrada.



Ilustración 33. Flujo de audio entrante

De las gráficas anteriores podemos extraer la siguiente información:

- packetsReceived/s representa el número de paquetes que se reciben cada segundo, cuya media es de 49.98 paquetes/s.
- packetsLost/s tiene un valor constante de 0, lo que indica que no se ha perdido ningún paquete de los enviados por el usuario local.
- Jitter marca un retardo constante de 0.002 ms.
- bytesSent/s y packetsSent/s (remoto) indican que el otro usuario remoto envió datos.

El parámetro audio/opus - SSRC 2917123791 representa el flujo audio de salida.

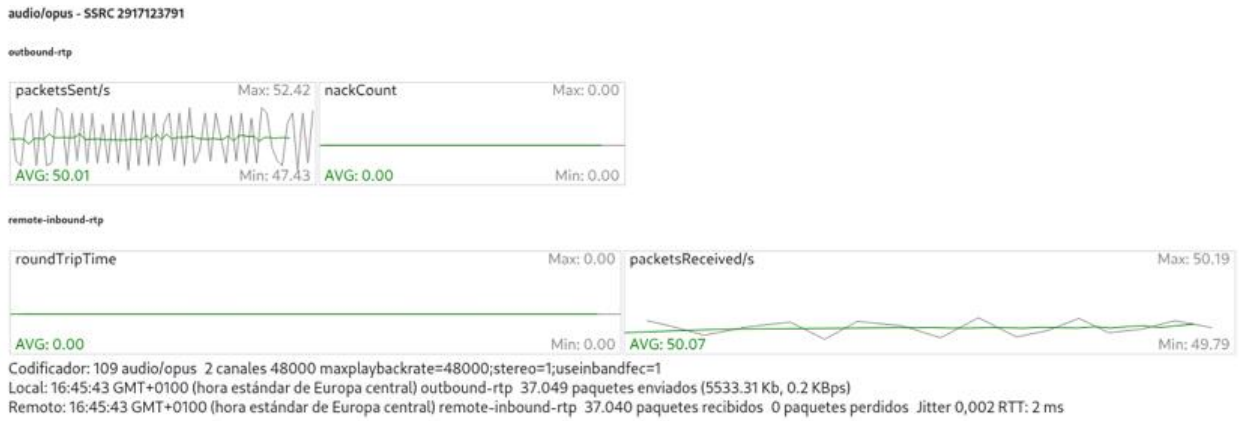


Ilustración 34. Flujo de audio saliente

De estas gráficas podemos extraer la siguiente información:

- packetsSent/s representa el número de paquetes de audio que se envían cada segundo, cuya media es de 50.01 paquetes/s.
- nackCount indica en número de paquetes que se han solicitado nuevamente porque no llegaron correctamente. Su valor siempre es 0.
- packetsRecived/s (remoto) indican número de paquetes recibidos en remoto.

En cuanto a las estadísticas de video se han obtenido los siguientes datos.

Video/VP8 – SSRC 1786203515 representa el flujo de video de entrada.

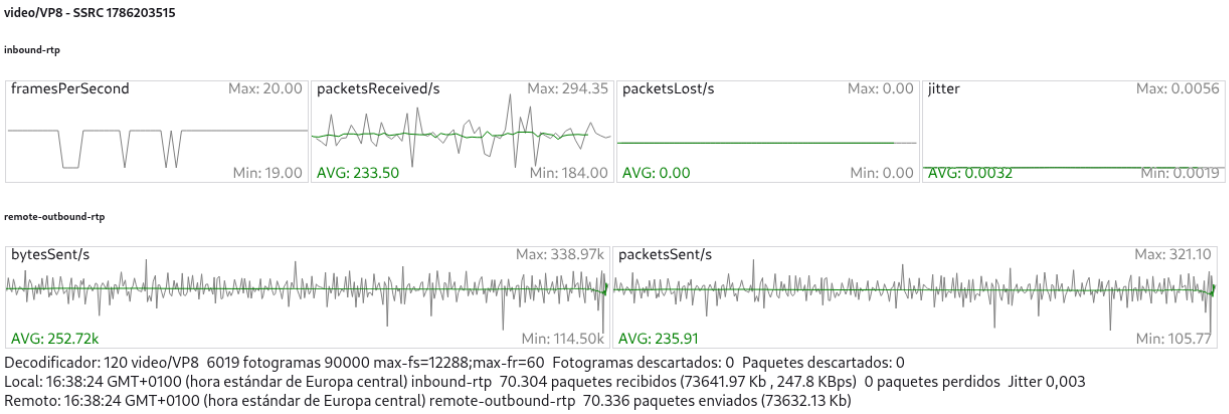


Ilustración 35. Flujo de video saliente

- packetsReceived/s: Se reciben una media de 233.5 paquetes de video por segundo.
- packetsLost/s tiene un valor constante de 0, no se ha perdido ningún paquete.
- Jitter marca un retardo constante de 0.0032 ms.
- bytesSent/s y packetsSent/s (remoto) indican que el otro usuario remoto envió datos.

Video/VP8 – SSRC 2212870151 describe el representa el flujo de video de entrada.

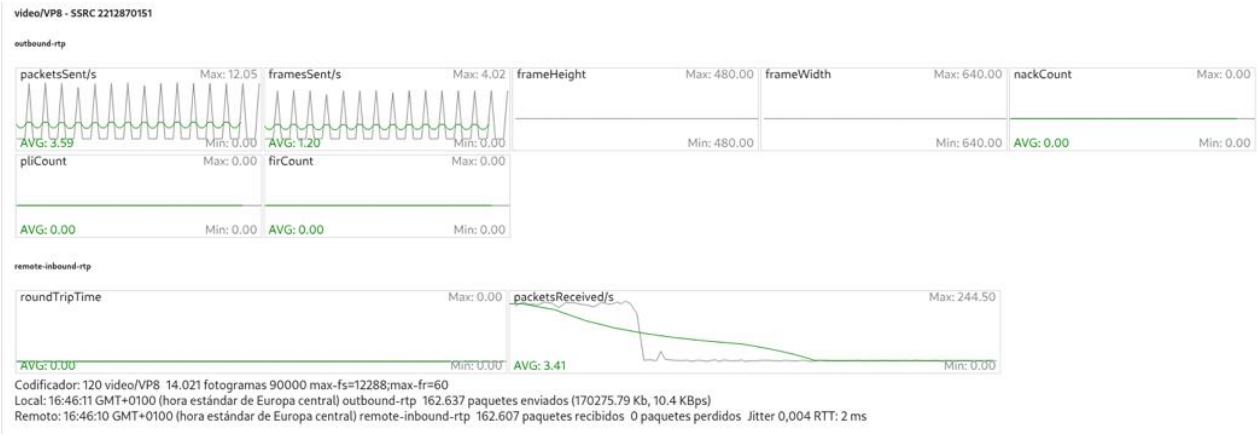


Ilustración 36. Flujo de video saliente

De estas gráficas podemos extraer la siguiente información:

- packetsSent/s marca un valor medio de 3.59 paquetes/s enviados.
- nackCount indica que ningún paquete se ha solicitado nuevamente.